

# Taller 5 — Introducción a Git

CE5507 · Modelado Software Hardware Orientado a Objetos · I Semestre 2026

Instituto Tecnológico de Costa Rica

Estudiantes: SebasHT15 · EmilioTec10

---

## Sección A: Investigación

### 1. Control de versiones

El control de versiones (también llamado control de código fuente o VCS, por sus siglas en inglés *Version Control System*) es un sistema que registra los cambios realizados sobre uno o más archivos a lo largo del tiempo, de modo que sea posible recuperar versiones específicas en cualquier momento. Permite a equipos de trabajo colaborar de forma ordenada, rastrear quién realizó cada modificación, cuándo la hizo y por qué, y revertir errores sin perder trabajo previo. Git es el sistema de control de versiones distribuido más utilizado actualmente, donde cada participante tiene una copia completa del historial del proyecto.

### 2. Repositorio

Un repositorio (o *repo*) es la estructura de almacenamiento que Git utiliza para guardar el historial completo de un proyecto: todos sus archivos, directorios, commits, ramas y metadatos asociados. Puede ser **local** (en la máquina del desarrollador) o **remoto** (alojado en plataformas como GitHub, GitLab o Bitbucket). En este taller se utilizó un repositorio remoto en GitHub llamado **Taller5-Modelacion**, creado por EmilioTec10 y compartido con SebasHT15 como colaborador.

### 3. Commit

Un *commit* es una instantánea (*snapshot*) del estado del repositorio en un momento dado. Cada commit queda identificado por un hash SHA-1 único (por ejemplo, **d75634e** para el commit inicial de este proyecto), incluye un mensaje descriptivo escrito por el autor, la fecha y hora exactas, y una referencia al commit anterior, formando así una cadena de historial. Los commits permiten entender la evolución del proyecto y volver a cualquier punto del pasado.

### 4. Conflicto (contexto de versionamiento)

Un conflicto de versiones ocurre cuando dos o más colaboradores modifican las **mismas líneas de un mismo archivo** en ramas o copias locales distintas, y Git no puede decidir automáticamente cuál versión conservar al intentar unirlos. Git señala el conflicto insertando marcadores especiales en el archivo afectado (`<<<<<<, =====, >>>>>>`) y el equipo debe resolverlo manualmente, eligiendo qué cambios mantener antes de completar el merge.

### 5. Rama (branch)

Una rama es una línea de desarrollo independiente dentro de un repositorio. La rama principal suele llamarse **main** (o *master* en proyectos más antiguos). Crear una rama permite trabajar en nuevas funcionalidades o experimentos de forma aislada, sin afectar el código estable. Una vez que los cambios están listos y probados, la rama se integra de vuelta a la principal mediante un proceso llamado **merge**. En

este taller se creó la rama **version-alternativa** y la rama de trabajo colaborativo **SmartStudy**.

## 6. Archivo Markdown (.md)

Markdown es un lenguaje de marcado ligero creado por John Gruber en 2004, cuyo objetivo es permitir escribir texto con formato utilizando una sintaxis sencilla y legible en texto plano. Los archivos con extensión **.md** se renderizan automáticamente en GitHub, convirtiéndose en documentación formateada con encabezados, listas, énfasis, tablas y enlaces. Son ampliamente usados para archivos README, wikis de proyectos y documentación técnica. En este taller se creó un archivo Markdown para documentar el sistema propuesto *SmartStudy*.

## Sección B: Exploración del Historial (Parte 5)

A continuación se analiza el historial de commits del repositorio **Taller5-Modelacion** en GitHub, con base en las capturas de pantalla de las tres ramas: **main**, **SmartStudy** y **version-alternativa**.

### a) Cambios realizados

| Hash    | Mensaje del commit                                                | Rama(s)                                   |
|---------|-------------------------------------------------------------------|-------------------------------------------|
| d75634e | Initial commit                                                    | main                                      |
| b146c72 | Se agrego pasatiempo.txt                                          | main                                      |
| 259db39 | Mis pasatimepos                                                   | main<br>version-alternativa<br>SmartStudy |
| 968ce0e | Se agregaron los pasatiempos de Emilio                            | main<br>version-alternativa<br>SmartStudy |
| 098ffa6 | Merge branch 'main' (1er merge)                                   | main<br>version-alternativa<br>SmartStudy |
| aa3712d | Se agregaron los pasatiempos comunes                              | main<br>version-alternativa<br>SmartStudy |
| 444bad0 | Merge pull request #1<br>from EmilioTec10/version-alternativa     | main<br>SmartStudy                        |
| 727009e | Se agrego nombre, descripcion<br>y usuario objetivo de smartstudy | main<br>SmartStudy                        |
| aa403b3 | Merge branch 'main'<br>(2do merge SmartStudy→main)                | main<br>SmartStudy                        |
| 31d3f62 | Se agrego la descripcion<br>y usuario final del sistema           | main<br>SmartStudy                        |

|         |                                                      |                    |
|---------|------------------------------------------------------|--------------------|
| de3617d | entradas del sistema SmartStudy                      | main<br>SmartStudy |
| f42335c | agregado salidas SmartStudy                          | main<br>SmartStudy |
| 33dc6fc | Merge pull request #2<br>from EmilioTec10/SmartStudy | main               |
| 6595dfd | Se agrego la reflexion<br>de la parte 6 del taller   | main               |

## b) Autor de cada cambio

El historial muestra la participación de dos colaboradores:

- **EmilioTec10:** realizó el Initial commit (d75634e), agregó pasatiempo.txt (b146c72), sus propios pasatiempos (968ce0e), los pasatiempos comunes (aa3712d), los merges con version-alternativa (444bad0) y SmartStudy (33dc6fc), y los commits de contenido sobre el sistema SmartStudy (727009e, 31d3f62, 6595dfd).
- **SebasHT15:** realizó el commit de sus pasatiempos (259db39) y los commits de entradas (de3617d) y salidas (f42335c) del sistema SmartStudy.

## c) Resolución del conflicto

El historial evidencia la resolución de un conflicto de versiones. Antes del commit **098ffa6** (*Merge branch 'main'*), ambos colaboradores habían modificado el archivo pasatiempos.txt de forma simultánea sin hacer pull previo: SebasHT15 agregó sus pasatiempos (259db39) y EmilioTec10 los suyos (968ce0e). Al intentar EmilioTec10 hacer push tras el commit de SebasHT15, Git detectó el conflicto. La resolución se realizó con GitHub Desktop, integrando ambas versiones en el commit de merge 098ffa6, tras el cual se agregaron los pasatiempos comunes en aa3712d.

## d) Uso de ramas

El repositorio utilizó tres ramas durante el desarrollo:

- **main:** rama principal. Contiene el historial completo, incluyendo los merges de las otras dos ramas.
- **version-alternativa:** creada para trabajar en paralelo sobre los pasatiempos comunes. Fue integrada a main mediante el Pull Request #1 (commit 444bad0).
- **SmartStudy:** creada para desarrollar el sistema propuesto en el ejercicio final. Fue integrada a main mediante el Pull Request #2 (commit 33dc6fc).

# Sección C: Reflexión (Parte 6)

## 1. ¿Qué ventajas tiene el uso de control de versiones?

El control de versiones ofrece múltiples ventajas para el desarrollo de software colaborativo:

- **Historial completo y auditable:** cada cambio queda registrado con autor, fecha y descripción, facilitando la trazabilidad del proyecto.
- **Reversibilidad:** es posible regresar a cualquier versión anterior del código en caso de errores o cambios no deseados.
- **Trabajo en paralelo sin interferencias:** el uso de ramas permite que múltiples desarrolladores trabajen simultáneamente en distintas funcionalidades sin afectarse mutuamente.
- **Integración controlada de cambios:** los merges y pull requests permiten revisar y discutir cambios antes de incorporarlos a la rama principal.
- **Respaldo distribuido:** en un sistema como Git, cada clon del repositorio contiene el historial completo, reduciendo el riesgo de pérdida de datos.

## 2. ¿Qué dificultades encontraron durante el proceso?

Durante la realización del taller se identificaron las siguientes dificultades principales:

- **Sincronización entre colaboradores:** coordinar quién hacía push primero y quién debía esperar resultó fundamental para evitar conflictos no planificados fuera del ejercicio guiado.
- **Comprensión del flujo de ramas:** entender cuándo hacer merge desde la rama secundaria hacia main y cuándo en sentido contrario requirió práctica inicial.
- **Mensajes de commit descriptivos:** mantener mensajes claros y consistentes desde el inicio del trabajo fue un hábito que se fue consolidando a lo largo del taller.

## 3. ¿Qué ocurrió durante el conflicto y cómo lo resolvieron?

Siguiendo las instrucciones de la Parte 3 del taller, ambos integrantes modificaron el archivo `pasatiempos.txt` de forma simultánea en sus copias locales sin hacer pull previo. SebasHT15 realizó su commit (259db39) y push exitosamente. Cuando EmilioTec10 intentó hacer push de su commit (968ce0e), GitHub rechazó la operación porque el historial remoto había avanzado. Git detectó que las mismas líneas habían sido modificadas por ambos.

La resolución se llevó a cabo utilizando GitHub Desktop: la herramienta mostró visualmente las secciones en conflicto del archivo, permitiendo elegir qué cambios conservar de cada versión. Tras seleccionar y combinar los cambios de ambos colaboradores, se completó el merge con el commit **098ffa6**. Posteriormente, en el commit **aa3712d**, se agregaron los pasatiempos que ambos tenían en común, completando la integración.

## 4. ¿Qué utilidad tienen las ramas en un proyecto real?

En un proyecto real de desarrollo de software, las ramas son fundamentales por varias razones:

- **Aislamiento de funcionalidades:** cada nueva característica o corrección de error (*feature branch*, *bugfix branch*) se desarrolla de forma aislada, sin comprometer la estabilidad de la rama principal.
- **Flujos de trabajo estructurados:** metodologías como Git Flow o GitHub Flow definen ramas específicas para producción, desarrollo y lanzamientos, aportando orden al ciclo de vida del software.
- **Revisión de código (*code review*):** los pull requests entre ramas permiten que otros miembros del equipo revisen, comenten y aprueben cambios antes de integrarlos, mejorando la calidad del código.
- **Experimentos sin riesgo:** es posible crear ramas experimentales para probar ideas nuevas sin afectar el código en producción; si el experimento falla, simplemente se descarta la rama.

- **Despliegues independientes:** en entornos de integración continua (CI/CD), las ramas pueden asociarse a ambientes específicos (desarrollo, staging, producción), facilitando el control de qué código se despliega en cada entorno.